01 K8s Pods basics and prep

Kubernetes Pods: Interview Quick Reference

What is a Pod?

Definition: A Pod is the smallest deployable unit in Kubernetes that wraps one or more containers.

Key Analogy:

- **Pod = Virtual Machine**
- **Containers inside Pod = Processes on that VM**

Critical Facts:

- Each Pod gets ONE IP address (not each container)
- Containers in a Pod share `localhost` - communicate via `localhost:<port>`
- All containers in a Pod run on the SAME node (cannot span nodes)
- Pods are ephemeral (disposable and replaceable)

Pod YAML Structure

```yaml
apiVersion: v1                # API version
kind: Pod                     # Resource type
metadata:                     # Identification
  name: my-pod
  labels:
    app: web
spec:                         # Specifications
  containers:
  - name: nginx
    image: nginx:1.21
```

Four Required Sections: apiVersion, kind, metadata, spec

Common Interview Questions

Q1: Why do we need Pods? Why not just use containers?

Answer:

- Pods provide an abstraction layer for orchestration
- Enable co-location of tightly coupled containers (app + sidecar)

- Provide shared resources (network, IPC) for helper patterns
- Simplify scheduling - all containers scheduled together

Q2: Can a Pod have multiple containers?

Answer: Yes, but typically:

- **ONE** application container
- **Additional** containers are helpers/sidecars (monitoring, logging, proxies)

Rule of thumb: If you remove container A and container B still makes sense independently, they should be in separate Pods.

Example - Good:

```
Pod: web-app + monitoring-agent
(Monitoring agent is useless without the app)
```

Example - Bad:

```
Pod: order-service + product-service
(Both are independent services - should be separate Pods)
```

Q3: What are common Pod statuses?

Status	Meaning	Action
Pending	Not scheduled yet or image pulling	Check events: `kubectl describe pod`
ContainerCreating	Pulling image/creating container	Wait, or check events if stuck
Running	All good, Pod is running	Desired state
ImagePullBackOff	Cannot pull image	Check image name/tag, registry access
CrashLoopBackOff	Container starts then crashes	Check logs: `kubectl logs pod`
Completed	Task finished successfully	Normal for Jobs
Error	Container exited with error	Check logs and events

Q4: What is the difference between `command` and `args`?

Docker terminology vs **Kubernetes terminology**:

Docker	Kubernetes	Purpose
ENTRYPOINT	command	Main executable
CMD	args	Arguments to executable

Example:

```yaml
spec:
  containers:
  - name: ubuntu
    image: ubuntu
    command: ["/bin/sh"]       # Override ENTRYPOINT
    args: ["-c", "echo hello"] # Override CMD
    # Executes: /bin/sh -c "echo hello"
```

Q5: What are the three restart policies?

```yaml
spec:
  restartPolicy: Always  # Default
```

Policy	Behavior	Use Case
Always	Always restart (even on success)	Web servers, long-running services
OnFailure	Restart only on error	Batch jobs, tasks
Never	Never restart	One-time tasks

Q6: How do containers in a Pod communicate?

Answer: Via `localhost` (they share the network namespace)

```yaml
spec:
  containers:
  - name: nginx
    image: nginx
    # Listens on port 80

  - name: app
    image: myapp
    # Accesses nginx via localhost:80
```

Inside the app container:

```shell
curl localhost:80  # Reaches nginx
```

Important: Containers CANNOT use the same port (will conflict)

Q7: What are common multi-container patterns?

1. Sidecar Pattern (most common):

```yaml
# Application + Monitoring Agent
containers:
- name: web-app
- name: monitoring-sidecar
```

2. Ambassador Pattern:

```yaml
# Application + Proxy
containers:
- name: app
- name: proxy-sidecar
```

3. Adapter Pattern:

```yaml
# Application + Log Formatter
containers:
- name: app
- name: log-adapter
```

Q8: How do you debug a failing Pod?

Three-step debugging process:

```shell
# 1. Check Pod status
kubectl get pods

# 2. Get detailed information and events
kubectl describe pod <pod-name>

# 3. Check container logs
kubectl logs <pod-name>
kubectl logs <pod-name> -c <container-name>  # Multi-container
```

Specific scenarios:

ImagePullBackOff:

```shell
kubectl describe pod <name> | grep -A 5 Events
# Look for: "Failed to pull image"
# Fix: Correct image name/tag or add imagePullSecrets
```

CrashLoopBackOff:

```shell
kubectl logs <pod-name> --previous  # See why it crashed
# Look for: application errors, missing env vars
```

Pending:

```shell
kubectl describe pod <name> | grep -A 5 Events
# Look for: "insufficient cpu/memory"
# Fix: Add nodes or reduce resource requests
```

Essential kubectl Commands

Basic Operations

```shell
# Create Pod
kubectl create -f pod.yaml
kubectl apply -f pod.yaml

# Delete Pod
kubectl delete pod <pod-name>
kubectl delete -f pod.yaml

# List Pods
kubectl get pods
kubectl get pods -o wide  # Show IPs and nodes
```

Debugging

```shell
# Detailed Pod info
kubectl describe pod <pod-name>

# View logs
kubectl logs <pod-name>
kubectl logs <pod-name> -c <container>  # Specific container
kubectl logs <pod-name> --previous      # Previous instance
kubectl logs -f <pod-name>              # Follow/stream

# Execute commands
kubectl exec <pod-name> -- <command>
kubectl exec -it <pod-name> -- /bin/bash  # Interactive shell

# Multi-container specific
kubectl exec -it <pod-name> -c <container> -- /bin/bash
```

Labels

```shell
# Show labels
kubectl get pods --show-labels

# Filter by labels
kubectl get pods -l app=web
kubectl get pods -l app=web,env=prod
kubectl get pods -l 'env in (dev,staging)'
kubectl get pods -l env≠prod
```

Debugging Access

```shell
# Port forward (for testing only)
kubectl port-forward <pod-name> 8080:80
# Access at localhost:8080
```

Important Configuration Fields

Environment Variables

```yaml
spec:
  containers:
  - name: app
    image: myapp
    env:
    - name: DATABASE_URL
      value: "postgres://db:5432"
    - name: LOG_LEVEL
      value: "debug"
```

Ports

```yaml
spec:
  containers:
  - name: nginx
    image: nginx
    ports:
    - name: http
      containerPort: 80
      protocol: TCP
```

Termination Grace Period

```yaml
spec:
  terminationGracePeriodSeconds: 30  # Default
  # How long to wait before SIGKILL after SIGTERM
```

Labels (Critical for Selection)

```yaml
metadata:
  labels:
    app: web-server
    version: "1.0"
    environment: production
```

Common Gotchas & Best Practices

❌ Anti-Patterns

1. **Using `latest` tag**

```yaml
image: nginx:latest  # BAD - unpredictable
image: nginx:1.21    # GOOD - specific version
```

2. **Multiple independent apps in one Pod**

```yaml
# BAD - these should be separate Pods
containers:
- name: order-service
- name: product-service
```

3. **No labels**

```yaml
# BAD
metadata:
  name: pod1

# GOOD
metadata:
  name: web-frontend-v1
  labels:
    app: web-frontend
    version: "1.0"
```

4. **Managing Pods directly in production**

- In production, use **Deployments**, not bare Pods
- Pods are building blocks, not production resources

Best Practices

1. **Always use specific image tags**
2. **Add meaningful labels**
3. **Set appropriate restart policies**
4. **One application per Pod**
5. **Use higher-level controllers (Deployments) in production**

Quick Troubleshooting Cheatsheet

Problem	Check	Command
Pod not starting	Events	`kubectl describe pod <name>`
Container crashing	Logs	`kubectl logs <name> --previous`
Can't pull image	Image name/registry	`kubectl describe pod <name>`
Pending forever	Resources	`kubectl describe nodes`
Wrong output	Env vars/config	`kubectl exec <name> -- env`
Network issue	Ports/connectivity	`kubectl exec <name> -- netstat -tuln`

Interview Pro Tips

When asked "What is a Pod?"

Start with: "A Pod is the smallest deployable unit in Kubernetes. Think of it as a wrapper around one or more containers that share the same network namespace and can share storage volumes."

When asked "Why Pods instead of containers?"

Explain the **sidecar pattern**: "Pods enable the sidecar pattern where you can co-locate tightly coupled containers. For example, your application container with a logging agent or monitoring sidecar that needs localhost access."

When asked about debugging

Show you know the **three-step process**: "First, I check Pod status with `kubectl get pods`. Then I use `kubectl describe pod` to see detailed events and identify which container is failing. Finally, I check logs with `kubectl logs` to see the actual error messages."

Common follow-up questions to prepare for:

- How do Pods get IP addresses? (CNI plugins)
- What happens when a Pod dies? (It's gone; use Deployments for recreation)
- How do Pods communicate across nodes? (Pod network via CNI)
- What's the difference between a Pod and a Deployment? (Deployment manages ReplicaSets which manage Pods)

Memory Hooks

Pod = Smallest Unit (like atom in chemistry)

Pod = VM, Containers = Processes (easiest mental model)

One App per Pod (except helpers/sidecars)

Describe → Logs → Exec (debugging order)

Labels = SQL WHERE (for querying/filtering)

Port-forward = Dev only (not production)

CrashLoop = Started then died (check logs)

ImagePull = Can't download (check image name)

Pending = Not scheduled (check resources)